MODULE 01 / TEAMMATE FIELD GUIDE

JD Analytics

Turn a job description into a reviewed list of what the employer asks for. Keep the exact words behind every requirement, and hand that saved list to Skill Matching.

JD | Resume | Profile | Talent Check | Match + GPS

The shared journey; each module also runs independently.

You receive	You return
A PDF/DOCX job description, or pasted JD text. Optional, explicit permission to send this input's text to Groq.	A saved Extraction with source_type=jd: company, role, skills, requirement groups, source evidence and review state.

Your responsibility

Participant Brief p.7 calls this LLM EXTRACTION. PDF/DOCX input, structured skills and the 12-category mapping are part of the core requirement. Pasted input and ProofPath source/importance controls extend it.

A successful handoff looks like this

A teammate can upload two different JDs, inspect/correct the skills, save a review, reload it and explain why the two jobs ask for different things.

> **Start from the working baseline**
>
> Assume YOU own this module. Run and understand the supplied working source, finish or improve your part, and test it. Push the verified working copy to YOUR OWN GitHub repository. Return its URL, commit SHA, actual test results and known limitations to the Scrum Master. No central merge or PR is required from you.

Before your module	After your module
The candidate supplies a JD. You do not need a resume, profile or company benchmark to run your module.	Module 5 receives the saved JD ID and revision. Its matcher loads the reviewed skills and groups; it does not scrape your screen.

Read in order: 2 build plan | 3 screen layout | 4 API | 5 logic | 6 integration | 7 tests | 8 presentation | 9 exact field reference

02 / IMPLEMENTATION PLAN

What to do, in order.

Keep changes inside your feature folder. Each step has an observable result; do not begin by replacing the whole app.

01 Trace one JD

Open the example JD and your service/router/UI files. Follow text -> extraction -> review -> saved revision -> matcher. Write down each boundary before editing.

02 Keep both inputs

Retain PDF/DOCX upload and Paste text. Reuse the shared document reader; do not write a second upload store or parser in the frontend.

03 Extract proposals

Keep JD-specific instructions in this module. Ask for real requirements, unfamiliar technologies, importance and relationships. Reuse the shared Groq schema/transport.

04 Ground the output

Every document skill and group needs a quote that exists in the source. Keep page/span links. Do not accept a fabricated source, model-owned ID or proficiency value.

05 Make review usable

Let the reviewer edit names/categories, add/remove skills and repair groups. Confirm unreviewed claims, group meanings and unparsed coverage explicitly.

06 Save one contract

PATCH the full current Extraction, not a reduced custom shape. Keep its ID, document ID and current revision. Cache success before refreshing the workspace.

07 Prove the handoff

Give Module 5 a saved, reviewed JD. Compare its group list with the original text, then run the feature and integration checks on page 7.

Files to open first

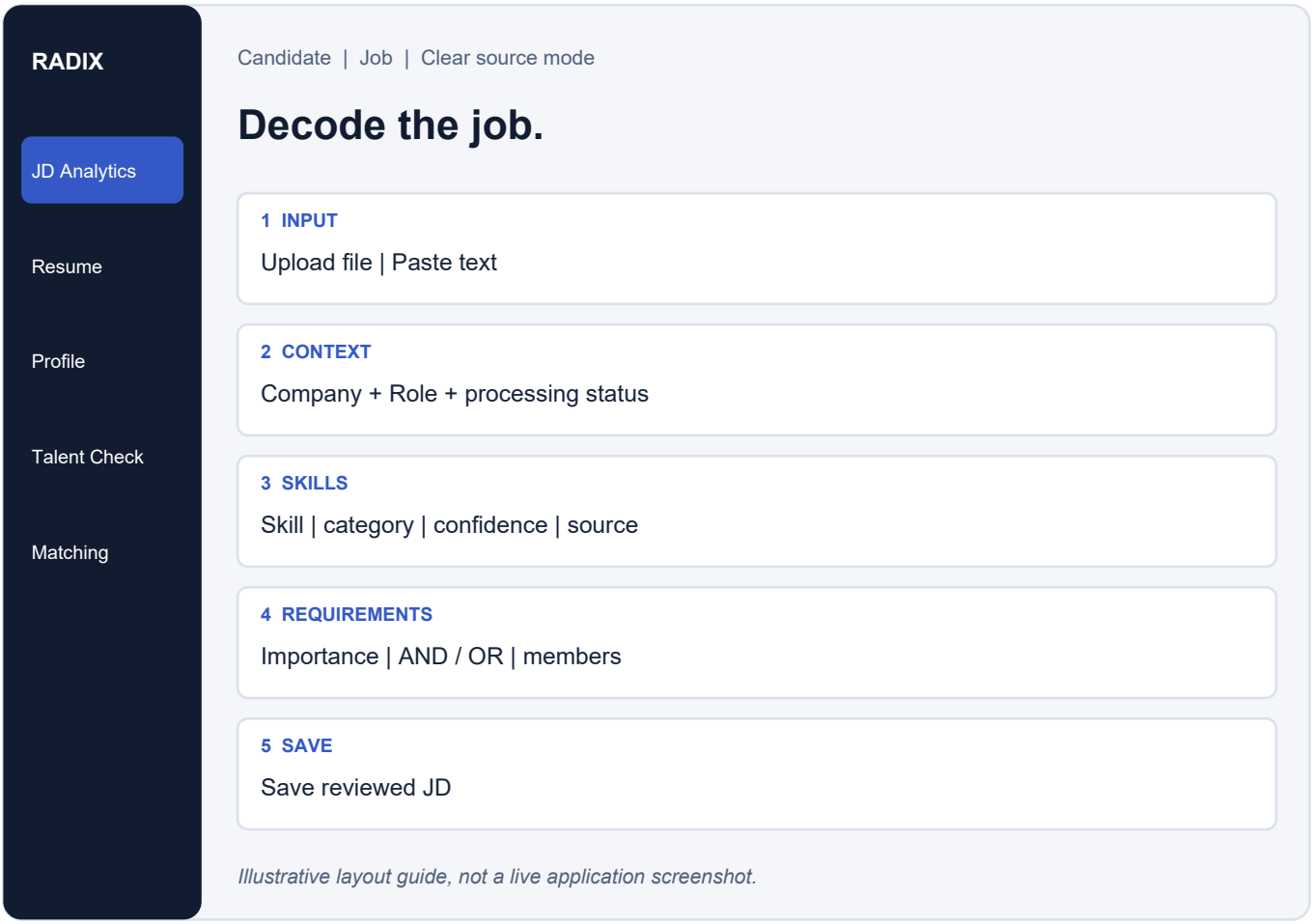
All paths below are relative to jd_analytics/.

File	What it owns
backend/service.py	Feature-specific extraction instructions, local rules and requirement grouping.
backend/router.py	Composes extraction routes and owns the pasted-text endpoint.
frontend/JDAnalyticsPage.tsx	Upload/paste flow, skill review and saved JD state.
frontend/RequirementReview.tsx	Importance, AND/OR, group members and review controls.
frontend/PasteJD.tsx	Text input, source name and per-input consent.

03 / SCREEN AND USER EXPERIENCE

What it should look like.

Keep the current app shell, navigation, typography and shared controls. The drawing shows information order, not a redesign request.



Screen state	Expected visible behavior
Nothing uploaded	Explain upload/paste and show no invented results.
Extracting	Disable duplicate submit; keep entered text until a result is accepted.
Model fails	Show Model unavailable / rules-only fallback; retain source and review warnings.
Save fails / 409	Keep the draft. For a stale revision reload and reconcile; do not silently overwrite.

Visual rules: 16 px body/form text, 14 px minimum supporting labels; clear headings; 44 px touch targets. Navy #111C30, blue #3459C6, white cards and #F4F6FA canvas. Use the existing Inter web font. On mobile, stack panels and keep actions readable. Do not rely on color alone for status.

04 / EXACT API BOUNDARY

Input, output and format.

Use the current Pydantic/OpenAPI contract. A UI label can change; an agreed JSON field cannot be renamed independently.

Endpoint	Body or result
POST /api/jd/extract	Multipart: file; hosted_consent=false by default.
POST /api/jd/extract-text	JSON: text, source_name, hosted_consent.
GET /api/jd[/ID]	List/load the current user's saved JD extractions.
PATCH /api/jd/ID	Full Extraction, current revision, review_status=reviewed.
DELETE /api/jd/ID	Remove extraction; original deletion is separate.

Accepted pasted-JD request; this is complete JSON

```
{
  "text": "Required: Python and SQL",
  "source_name": "Synthetic JD",
  "hosted_consent": false
}
```

Current synthetic response excerpt

Only selected fields are shown below. Page 9 describes the complete shape. Snapshot UUIDs are teaching examples, not IDs to reuse in a running app.

```
{
  "id": "8ff95599-3f83-5a69-993f-7a5df3b52e2e",
  "revision": 2,
  "source_type": "jd",
  "source_file": "junior-data-engineer-jd-SYNTHETIC.docx",
  "company": "Synthetic Data Co",
  "role": "Junior Data Engineer (synthetic)",
  "review_status": "reviewed",
  "extraction_mode": "local_limited",
  "llm_status": "rules_only",
  "document_id": "d985df0b-a014-55e4-82cb-534751d5da65"
}
```

Every mutation needs X-ProofPath-Client: web (or cli). Cloud requests also need the current user's Bearer JWT. The shared web helper supplies these headers. File upload uses multipart; never manually set its boundary. JSON fields use snake_case.

05 / REASONING AND BUSINESS RULES

A skill list is not enough. Preserve the relationship.

Read responsibilities and qualifications as requirements. Company names and job titles provide context; words in those fields alone are not required skills. A named technology outside the old vocabulary still belongs in the reviewed result.

Situation	Exact expected reasoning
Required: Python and SQL	One mandatory AND group. Both members matter; each receives half of that group's potential coverage.
Python and either PyTorch or TensorFlow	Two mandatory groups: Python alone (AND), plus PyTorch/TensorFlow (OR). Do not make all three mandatory.
Docker would be useful	Preferred group, not mandatory. Use the source wording and context.
Rust / Apache Arrow is unfamiliar	Groq may propose it with checked evidence; rules may miss it. Keep missing-coverage warnings until a human corrects them.
No Java required	Negated/contextual wording must not become a positive compulsory requirement. Keep uncertainty visible for review.

Important edge case

The group contract supports flat AND/OR groups. Mixed statements need a correct decomposition or a requires_review flag. Never invent nested-group fields without a team-wide schema change.

The LLM path

Document reader -> rules as hints -> consented Groq strict JSON -> schema/quote checks -> server-owned IDs -> editable proposals -> reviewed save. Model claims start needs_review; verification stays unverified. One bounded regeneration is allowed for invalid output. Provider errors produce an honest fallback by default.

Never cross this boundary

No numeric candidate skill levels. No scores computed here. Confidence means confidence in extraction, not candidate ability. No private document leaves the backend for Groq without the input's consent.

06 / INTEGRATION AGREEMENT

Own it. Test it. Publish your part.

Your task is a working module on your own GitHub. Keep the supplied shared contracts and exact output formats so the Scrum Master can integrate the returned code.

Team rule	What you do
Service boundary	backend/service.py: extract(sections, filename, hosted=False) -> Extraction. Input sections are page/text dictionaries; the route supplies document ownership and persistence.
One feature owner	Own jd_analytics/backend, frontend, tests and docs. Keep the common support area in your clone. Run your module independently before claiming it works.
One contract source	Read shared/contracts/models.py. OpenAPI and generated TypeScript follow it. Do not maintain a second handwritten schema or change categories alone.
One storage/auth path	Use shared store/document/auth helpers. Do not add a second database, anonymous cloud owner, public CV URL or frontend API secret.
One current record	Use actual returned IDs/revisions, owned by the signed-in user. Pass saved IDs to consumers; never hardcode teaching snapshot IDs.
Your GitHub	Create a private repository on YOUR account. Rename the old origin to scrum-master, set your new origin, review staged files and scan for secrets, then commit and push. See SUBMIT_YOUR_REPOSITORY.md.
Return working evidence	Fill DELIVERY.md with your URL, commit SHA, actual commands/results, three normal and three failure cases, synthetic screenshots and known bugs. If a check fails, fix it or report it honestly; do not copy passing counts.

The official category vocabulary

Every extracted skill has exactly one official code or OTHER. The extra descriptive skill families are filters, not replacement categories. Only the twelve official codes can have numeric ratings or company expectations.

Category	Category
COD - Coding	CLOUD - Cloud
DSA - Data Structures & Algorithms	SQL - SQL
OOD - Object-Oriented Design	SWE - Software Engineering
APTI - Aptitude	SYSD - System Design
COMM - Communication	NETW - Networking
AI - AI	OS - Operating Systems

Keep these meanings separate

Confidence = how clearly the document supports an extraction. Competency = explicit self-rating, null or 1-10. Review = a human checked the output. Verified = independently established; this app's claims remain unverified. Hypothetical = GPS preview only.

07 / RUN AND VERIFY

Your definition of done.

A module must run independently as well as inside the shared app. Record actual commands and failures; a screenshot or a copied passing count is not test evidence.

Use the existing project, not a second app

From the repository root, install the existing requirements/lockfiles if needed. Open two terminals. If the ports are already running, use them; do not kill unknown processes. Never overwrite an existing .env.

```
.\jd_analytics\run.ps1 api
# Separate terminal:
.\jd_analytics\run.ps1 web
# Real standalone API check:
.\jd_analytics\run.ps1 check
```

Clone the full repo and install root dependencies (README). Run these from its root. Open http://127.0.0.1:5173/; Load synthetic inputs supplies upstream fixtures. No key is needed for rules. A Groq key belongs only in the private root .env; restart Python after editing it.

Test	Pass means
Two different synthetic JDs, PDF + DOCX	Useful skills/groups appear; differences reflect source text, not hardcoded employer names.
Paste same source	Same contract and private source linkage; input_method=pasted_text.
Python AND an ML-library alternative	Two correct groups; alternatives are not all compulsory.
Groq off / denied / invalid quote	No unauthorized call; honest rules fallback or safe error; fabricated quote never accepted.
Manual correction + save/reload	Latest review revision is loaded by Module 5.
Empty/corrupt/scanned/large input	Useful error; no invented extraction. Existing input limits remain enforced.

Your focused commands

```
.\.venv\Scripts\python.exe -m pytest jd_analytics/tests -q
npm run test -- jd_analytics/frontend
```

Before returning: run typecheck, lint, build and the standalone browser check. Use the project virtualenv for Python. Keep assertions meaningful and record actual results.

Error behavior to demonstrate

422 = invalid/unreadable input; 413 = size limit; 415 = unsupported file; 404 = missing/unowned record; 409 = stale revision; 503 = unavailable storage/benchmark. Groq may fall back and return a successful Extraction with llm_status=unavailable instead of an HTTP error. Show the status, not just HTTP 200.

08 / LIVE DEMO AND PRESENTATION

Show it. Explain it. Hand it over.

Suggested speaking time: four minutes per module. Five modules use about 20 minutes, leaving 10 for the problem, architecture, challenge and close within the 30-minute presentation. Q&A is separate (15 minutes).

Reproducible synthetic demo

1. Open JD Analytics. Upload demo/final/junior-data-engineer-jd-SYNTHETIC.docx in rules mode for the fixed demo.

2. Point to Synthetic Data Co and Junior Data Engineer. Explain that the employer and document are invented fixtures.

3. Show five groups: Python+SQL, Pandas, Linux, Docker, and AWS OR GCP. The first three are mandatory; the final two preferred.

4. Open a source quote. Show how category and importance can be corrected, then save the reviewed JD.

5. Hand over the current JD ID/revision. If demonstrating Groq, use an authorized synthetic input and show the actual processing label; do not claim it ran during the fixed rules demo.

What to say and show

Time	Speaking cue
0:00-0:35	My module answers what this specific job asks for. It takes a JD, not a candidate, and creates structured requirements.
0:35-1:20	Show upload/paste, company/role and the skills. Explain one category and one original source quote.
1:20-2:15	Show importance and AWS OR GCP. Explain why either alternative can satisfy that group.
2:15-3:10	Explain Groq proposals, deterministic evidence checks and human review. Show the truthful current processing mode.
3:10-4:00	Save the review, name the JD ID/revision handoff and explain one failure state. Transition to resume parsing.

> **Transition to the next speaker**
>
> We now know what the job asks for. Module 2 will extract what the candidate's resume actually says.

Use this language honestly

Say which parts you implemented, adapted, tested or learned. If showing the working team baseline, call it the integrated team implementation. Do not claim a live model, official benchmark or feature history that has not actually been demonstrated.

09 / FIELD REFERENCE AND QUESTIONS

Keep this beside your editor.

These are exact field names from the current Python contracts. Defaults fill omitted optional fields; partial excerpts are not safe replacements for full PATCH payloads.

Group	Exact fields / meaning
Identity / version	schema_version, id, revision, source_type, source_file, document_id, input_method
Document context	company, role, name, email, sections
Content arrays	skills, requirement_groups, education, projects, experience, certifications, internships, hackathons, achievements
Lifecycle / processing	review_status, extraction_mode, parser_version, model_version, prompt_version, llm_status, llm_metadata, warnings
Each skill: identity	skill_id, skill_name, normalized_name, category_code, skill_family
Each skill: evidence	evidence, confidence, source_page, source_start, source_end, source_file, source_document_id, source_type, evidence_type
Each skill: state	verification_status=unverified; claim_status=claimed|needs_review; claim_basis=explicit|inferred
Each requirement group	requirement_id, skill_ids, operator=AND|OR, requirement_type=mandatory|preferred|unspecified, requires_review, evidence, source_page

Questions you should answer without memorizing code

Why an LLM? Natural prose and unfamiliar technology names exceed a fixed keyword list. Source checks and human review limit unsupported proposals.

Is LangChain mandatory? The intended module uses LLM extraction. The current implementation calls Groq directly; LangChain itself is not prescribed by the brief.

What if Groq fails? Return a labelled rules fallback, or a safe error if configured. Do not call rules results LLM processed.

Source of truth and current limits

Read your module's README.md, PLAN.md, CONTRACT.md and USAGE.md; shared/contracts/models.py; docs/BUILD_PLAN.md; docs/DEPLOYMENT_SETUP.md; docs/LLM_EXTRACTION_IMPLEMENTATION.md; verification/RESULTS.md; and docs/TEAM_PRESENTATION_CHECKLIST.md. Full current validated synthetic JSON snapshots are under data/synthetic/handoff/. Snapshot IDs are not live IDs.

Original Participant Brief: pp.7-10 (roles/contracts), p.11 (official test pack), p.13 (stack/done). Event Plan p.4: 30 minutes presentation + 15 Q&A. Original baseline suite (see new checkout results in verification/RESULTS.md): 134 backend, 17 frontend and 14 browser tests. Live Groq quality, cloud RLS/deployment and official organizer-pack/benchmark acceptance remain pending. All guide examples are synthetic.